Cost-Aware Multi-agent System for math problem

Alan Zhang zhangz68@myumanitoba.ca
Xinmeng Tan tanx3@myumanitoba.ca
Young Liu liuy32@myumanitoba.ca

1 Problem Statement

Large language models (LLMs) can achieve strong performance on many math word-problem benchmarks, but their multi-step reasoning can be unreliable on harder questions, even when using Chain-of-Thought prompting and human-inspired "plan–check–revise" workflows. At the same time, state-of-the-art multi-agent reasoning systems (e.g., planner → solver(s) → critic → judge) improve robustness by adding specialization and verification, but they also introduce substantial computational overhead because each additional agent typically implies another LLM call and additional prompt/completion tokens. In an undergraduate project setting, this cost barrier is especially important because advanced multi-agent features (e.g., monitor-based RAG and large-scale system engineering) are difficult to implement and expensive to run at scale.

Therefore, the central goal of this project is to reduce the average computational cost of multi-agent math reasoning while maintaining comparable accuracy. Concretely, we study whether a meta-planner with dynamic routing can avoid unnecessary agent calls on easy problems and reserve heavier multi-agent computation only for medium/hard problems. Our baseline is a fixed 3-agent pipeline (Solver → Critic → Judge) that is executed for every question. Our proposed system introduces an A-base agent, which is a single-pass solver that outputs (i) an initial answer and (ii) a rough difficulty estimate, which the router then uses as a signal to choose among multiple routes: it directly accepts A-base for easy cases, runs the baseline route for medium cases, or invokes a more involved planner-based route for hard cases.

To make the trade-off explicit and measurable, we define "cost" using four metrics: (1) number of LLM calls per problem, (2) total tokens per problem (prompt + completion), (3) approximate wall-clock time per problem, and (4) estimated API dollar cost per problem. We evaluate both systems on a challenging math dataset (OlympiadBench) and report accuracy together with these cost metrics, using consistent experimental settings across 150 questions. The key research question is:

> *Can a meta-planner and dynamic routing reduce the average computational cost compared with a fixed multi-agent pipeline, while keeping similar accuracy?*

Our intended outcome is an empirical, reproducible comparison between the fixed baseline pipeline and the dynamically routed multi-agent system, including clear evidence (tables/plots) of when multi-agent reasoning is "worth it" under realistic compute and budget constraints.

2 Proposed vs. Accomplished

- **Baseline system.** *Proposed:* Use an existing open-source multi-agent framework (Eigen-1 style) as the unmodified baseline, with fixed role prompts (Reviewer/Solver/Reasoner). *Accomplished:* Implemented a fixed 3-agent baseline pipeline (Solver → Critic → Judge) with exactly 3 calls per problem.

- **Core efficiency idea.** *Proposed:* Reduce cost mainly by prompt optimization/compression and by introducing efficiency strategies (dynamic activation, caching/reuse, tool-assisted verification, adaptive agent selection). *Accomplished:* Implemented a meta-planner + router that dynamically selects among routes, using an A-base agent to provide a quick answer and a difficulty signal; easy problems

take a cheaper route, while harder problems trigger heavier reasoning.

- **New agent design.** *Proposed:* Keep the overall architecture mostly unchanged and optimize prompts within the existing multi-agent framework. *Accomplished:* Added new components (A-base and meta-planner/router) and restructured the pipeline into multiple routes (direct / baseline / planner route), which is a significant design change from the proposal.

- **Efficiency tracking and cost definition.** *Proposed:* Implement an efficiency tracking module to record runtime, token count, and number of agent calls, and support batch processing. *Accomplished:* Defined cost using four metrics (LLM calls, total tokens, wall-clock time, and estimated API cost) and reported aggregate/average cost for baseline vs. advanced systems.

- **Dataset and sampling plan.** *Proposed:* Use a clustered sample of about 500 problems drawn from GSM8K, U-MATH, and HardMath, covering multiple difficulty levels. *Accomplished:* Switched to OlympiadBench (olympiad-level problems) and evaluated on 150 questions. This change was made to stress-test multi-agent reasoning on very challenging problems under explicit cost measurement.

- **Model and compute plan.** *Proposed:* Use a medium-sized API model (e.g., GPT-3.5 series) for controlled efficiency tests, and an open-source 4B model (Gemma) locally, with optional rented GPU. *Accomplished:* Used API models for experiments: baseline used GPT-4.1; advanced used a mix of GPT-4.1-mini (easy/medium) and GPT-4.1 (hard), to reduce dollar cost while maintaining performance.

- **Experimental scope.** *Proposed:* Run a preliminary experiment on 50 questions, then extend to the full sample if time allows. *Accomplished:* Ran the full evaluation on the selected 150-question set and reported overall accuracy and cost comparisons; some originally proposed optimizations (e.g., caching/tool-assisted verification) were deprioritized in favor of routing and model-mix efficiency.

3 Related work

Multi-agent systems (MASs) are a central place of studies of contemporary artificial intelligence. A multi-agent system includes a couple of decision-making agents that interact in shared surroundings to obtain common or conflicting desires (Gupta et al., 2022). In math problem solving, a common multi-agent instantiation is a debate-style pipeline in which two agents argue for/against the correctness of reasoning steps, and a judge selects the winner with a confidence score (Zhang and Xiong, 2025). In mathematical reasoning tasks, a common multi-agent paradigm separates solution generation and solution evaluation into distinct roles (e.g., a Solver and a Judge), forming a propose–solve–judge pipeline to improve solution quality (Chen et al., 2025). However, such multi-agent pipelines often apply the full interaction pattern uniformly, which can introduce substantial overhead, motivating cost-aware routing and scheduling to better balance accuracy and computation. Multi-agent designs may bring benefits in modularity and extensibility, but they can also incur extra communication and computation costs; therefore, practitioners often face a trade-off between scalability/extensibility and runtime efficiency (Gupta et al., 2022).

Beyond role-based multi-agent pipelines, a complementary line of work injects external evidence into LLM reasoning. Retrieval-augmented generation (RAG) combines a parametric language model with a non-parametric memory, typically by retrieving relevant documents and concatenating them with the prompt so that generation is grounded in external sources rather than relying solely on model parameters (Klesel and Wittmann, 2025). For mathematical question answering, RAG-style systems for conceptual question answering retrieve textbook or workbook passages to reduce hallucinations and improve answer quality, but empirical studies show a trade-off between responses that are tightly grounded in the retrieved materials and responses that are preferred by human learners (Levonian et al., 2023). Moreover, training-time approaches such as DeepSeekMath focus on domain-specific pre-training: they continue training a base model on a very large corpus of math-related text and code to substantially

boost mathematical reasoning ability without relying on external tools or retrieval at inference time (Shao et al., 2024). By contrast, our work introduces an A-base: a lightweight, per-problem shared workspace produced by a single pass agent that records a provisional answer, difficulty and confidence estimates, and a short summary, which subsequent agents can reuse as structured external context.

Beyond fixed solver–critic pipelines, another line of work studies meta-level agents that plan and design the overall multi-agent workflow, sitting above base agents to decide how tasks are decomposed, which tools or collaborators to invoke, and when to terminate the process. MACI (Chang, 2025) introduces a central planner that parses a user request into a dependency graph of subtasks, instantiates specialized agents for each role, and orchestrates their communication through a shared memory and coordination module. Moreover, MetaAgent (Zhang et al., 2025) further automates this process by synthesizing a finite-state-machine-based multi-agent system from a natural language task description, and then optimizing state transitions and tool usage so that the automatically generated workflows approach the performance of strong human-designed systems on software and machine-learning tasks. Other meta-control frameworks, such as HealthFlow for clinical EHR analytics with meta, evaluator, and reflector agents (Zhu et al., 2025) and Multi-Agent Evolve for RL-based training of solver–proposer–judge agents (Wang et al., 2024), focus on improving robustness, domain coverage, and self-improvement of agentic systems rather than explicitly controlling the accuracy–cost trade-off. Inspired by these meta-level designs, our meta-planner is deliberately lightweight: it only reads difficulty and confidence signals from a single A-base attempt and routes each math problem to a direct, baseline, or heavy multi-agent pipeline, aiming to adjust computational cost per question while keeping the underlying agents themselves simple.

Also, Recent work on multi-agent communication points out that noisy, unstructured messages and fragmented communication protocols can undermine coordination, and calls for more robust, interoperable messaging mechanisms (Chen et al., 2024; Derouiche et al., 2025). Motivated by this, we add a small summarizer agent in the heavy route that compresses multiple solver outputs into a compact, structured per-question summary, providing a more stable communication artifact for downstream agents while keeping the extra token and latency overhead modest.

4 Dataset

Our project evaluates multi-agent large language model (LLM) pipelines for Olympiad-level mathematical reasoning. We use OlympiadBench (He et al., 2024), a bilingual benchmark of Olympiad-level mathematics and physics problems released with data and evaluation code.[1]

In our experiments, we treat the dataset as test-only. We do not train on these problems, and we use the same fixed problem set to compare a baseline pipeline against our advanced dynamically routed pipeline described in later sections.

Task. Each example is an open-ended final-answer problem. The input is a problem statement written in natural language and LaTeX. The output is a single final answer string, such as an integer, an exact fraction, or a short list of values.

Evaluation subset used in this report. All experiments are conducted on a processed subset of 150 open-ended mathematics problems from OlympiadBench (file: `OE_MM_frontier_format_renumbered.json`). Each record contains four fields: `id`, `difficulty` (used as a math subfield label), `question`, and `answer`.

Late-stage text-only sanity check. During final validation, we found that many items in our 150-question evaluation set are diagram-dependent, but our runs did not include image inputs (only text and LaTeX were provided). To reduce this confound under the same time budget, we additionally evaluated a 100-question text-only subset (file: `OE_TO_textonly_100_frontier_format.json`) that excludes problems requiring figures. We report these supplementary results in Section 6.7.

[1] https://github.com/OpenBMB/OlympiadBench

Statistic	Value
Number of problems	150
Subfields Geometry	112
Combinatorics	28
Algebra	6
Number Theory	4
Questions containing diagrams	150
Questions containing LaTeX	112

Table 1: Dataset composition statistics for the 150-problem evaluation set.

Statistic	Value
Question length (words), mean	101.95
Question length (words), median	55
Question length (words), minimum	15
Question length (words), maximum	629
Question length (words), 90th percentile	347
Diagrams per question, mean	1.15
LaTeX commands per question, mean	5.27
LaTeX commands per question, maximum	36

Table 2: Question length and symbolic complexity statistics for the 150-problem evaluation set.

Answer format (approximate)	Count
Integer	69
Decimal number	2
Fraction	32
Ordered pair or tuple	2
Multiple values separated by commas	23
Other symbolic or algebraic expression	22

Table 3: Approximate distribution of answer formats in the 150-problem evaluation set.

Why this dataset is challenging. OlympiadBench is designed to be challenging even for strong large language models. In our setting, the task is difficult for several reasons. First, many problems rely on geometric diagrams for critical information, while our current experiments use a text-only representation. Second, the questions exhibit high symbolic density, frequently involving LaTeX expressions and multi-step constraints. Third, evaluation relies on strict final-answer matching, meaning that small formatting differences in exact expressions or multi-value answers can lead to incorrect automatic evaluation.

Example input and output. We show two shortened examples to clarify the input and output format.

Example 1 (Geometry). *Input:* "In a quadrilateral $PQRS$, find the perimeter."
Output: `52`

Example 2 (Geometry). *Input:* "Find the slope of segment AC."
Output: `-3/4`

4.1 Data preprocessing

We perform minimal preprocessing and do not alter the mathematical content of the original problems. Preprocessing is applied only to ensure compatibility with our solver and evaluation scripts. Specifically, we convert each item into a minimal record containing `id`, `difficulty`, `question`, and `answer`, renumber the problem identifiers to be contiguous from 1 to 150 for ease of logging, and retain diagram references implicitly without embedding image data.

4.2 Data annotation

This project does not introduce new human annotations. We directly use the ground-truth final answers provided with the dataset for evaluation. As a result, inter-annotator agreement is not applicable.

5 Baselines

Our project evaluates whether a dynamically routed multi-agent system can improve the efficiency–accuracy trade-off of math problem solving. To quantify any gains, we compare our advanced system against a fixed multi-agent baseline that follows the standard "generate–critique–select" pattern.

5.1 Baseline: Fixed 3-agent pipeline (Solver → Critic → Judge)

Motivation. A common multi-agent approach is to first generate a complete solution attempt and then add verification and selection stages to improve reliability. To obtain a clean and feasible baseline in an undergraduate setting, we implement a simplified but representative pipeline with fixed compute that is applied to every question.

Pipeline. For each problem, we always run three agents in sequence:

1. **Solver:** generates a full solution attempt and a final answer.
2. **Critic:** reviews the solver output, flags potential errors, and proposes corrections or an alternative answer when needed.
3. **Judge:** reads the problem together with the solver/critic outputs and selects the final answer to return.

Key properties.

- **Fixed compute:** exactly 3 LLM calls per problem.
- **Consistent setup:** the same model family and a similar prompt style are used across the three agents to reduce confounding factors.
- **Test-time only:** no fine-tuning is performed; performance differences come from the multi-agent structure and the number of calls.

Rationale. This baseline isolates the effect of adding critique and adjudication on top of a single solution attempt. It also provides a clear comparison point for our advanced system, whose main additional components are a front-end single-pass solver (A-base) and a router/meta-planner that allocates different amounts of computation depending on estimated difficulty.

6 Your approach

6.1 High-level idea

Our approach uses dynamic routing so that easy problems avoid unnecessary multi-agent computation, while harder problems receive additional guidance. The goal is to improve the efficiency–accuracy trade-off relative to the fixed baseline.

6.2 System overview

The advanced system adds a front-end single-pass solver (A-base) and a router. Given a question, A-base produces an initial answer and a coarse difficulty label. The router then chooses one of three routes:

1. **Direct route (easy):** return the A-base answer.
2. **Baseline route (medium):** run the fixed baseline pipeline from Section 5.
3. **Planner route (hard/very hard):** run a lightweight planner, then solve, and finalize with a summarizer.

6.3 Routing and added modules

Routing is a deterministic mapping from the A-base difficulty label to the route above. The planner route uses two additional components: SoftPlanner (a short high-level plan) and a Summarizer (standardized final answer format).

6.4 Model mix

To reduce dollar cost, the advanced system uses GPT-4.1-mini for easy/medium routes and GPT-4.1 for hard problems. The baseline uses GPT-4.1 only.

6.5 Evaluation summary

We evaluate on 150 OlympiadBench questions. Table 4 reports accuracy and cost metrics. The advanced system improves accuracy (30% to 36%) while reducing the average runtime (31.54 s to 25.17 s) and the estimated total cost (approximately $ 5 to $ 1).

Metric	Baseline	Advanced
Model setup	GPT-4.1 only	Mix of GPT-4.1-mini / GPT-4.1
Number of questions	150	150
Total tokens	785,042	1,237,331
Avg. tokens	5,233.6	8,248.9
Total runtime (s)	4,731.75	3,775.56
Avg. time (s)	31.54	25.17
Accuracy	30%	36%
Approx. $ cost*	$5	$1
Avg. $ cost	$0.0333	$0.0067

Table 4: End-to-end comparison on 150 OlympiadBench questions.

*Approximate cost computed from recorded token counts and the model pricing used at the time of the experiments.

6.6 Prompt revision for token efficiency

After the first round of experiments, we found that token usage was higher than expected, and the majority of tokens came from the input side. This indicates that prompt overhead (long system instructions and repeated formatting rules) can dominate the overall cost, especially when multiple agents are involved.

To reduce prompt and completion tokens, we audited our agent prompts and revised the YAML templates by removing redundant instructions, tightening output requirements (shorter JSON fields and stricter brevity constraints for verification stages), and making the routing policy more lightweight. We then re-ran the advanced system using the revised prompt template and evaluated the outputs with the same scoring script.

Table 5 shows that the revised prompt reduces total tokens while slightly improving accuracy. Total tokens decreased from 1,237,331 to 1,164,019 (a 5.9% reduction), and re-evaluated accuracy increased from 36.0% to 38.67%. Runtime remained similar (about 25 seconds per question). In the revised run, prompt tokens accounted

for 950,168 of 1,164,019 total tokens, confirming that prompt length is a key driver of cost in our pipeline.

Metric	Advanced (original prompt)	Advanced (revised prompt)
Number of questions	150	150
Total tokens	1,237,331	1,164,019
Avg. tokens / question	8,248.9	7,760.1
Total runtime (s)	3,775.56	3,817.55
Avg. time (s)	25.17	25.45
Accuracy (re-evaluated)	36.0%	38.67%

Table 5: Effect of prompt revision on the advanced system. The revised prompt reduces token usage and slightly improves accuracy, while runtime remains similar.

6.7 Supplementary results on a text-only subset (n=100)

Because the main 150-question set contains many diagram-dependent problems but our runs used text-only inputs, we ran a late-stage sanity check on a 100-question text-only subset. We report re-evaluated numeric accuracy using our post-hoc scorer, which normalizes common equivalent formats (e.g., fractions, roots, and exponent expressions).

Metric	Baseline (GPT-4.1)	Advanced (routed mix)
Number of questions	100	100
Total tokens	454,680	710,374
Avg. tokens / question	4,546.8	7,103.7
Total runtime (s)	2,369.40	2,576.63
Avg. time (s)	23.69	25.76
Accuracy (re-evaluated)	41.0%	47.0%

Table 6: Supplementary results on the 100-question text-only subset, evaluated with a re-evaluated numeric scorer.

Overall, the routed system still improves accuracy (+6.0 pp) but consumes more tokens per question due to routing and extra verification steps.

7 Error analysis

This section discusses where the baseline and our approach fail, and why the observed results should be interpreted cautiously.

7.1 Study limitation: small sample size

Our evaluation uses $n = 150$ questions, which is enough to observe qualitative trends but is not large enough to support strong conclusions about small accuracy differences. A change of only a few questions can shift accuracy by multiple percentage points. Therefore, we treat the current results as an initial empirical study rather than a definitive conclusion, and we focus on error patterns and actionable system issues.

7.2 Baseline weakness and "strategy-level" failures

The baseline (Solver $\rightarrow$ Critic $\rightarrow$ Judge) relies on a single Solver attempt. For hard problems, many failures appear to be strategy-level: if the Solver chooses an unproductive approach early, the Critic and Judge can sometimes fix local mistakes but rarely recover by switching to a different solution strategy. This makes the baseline relatively weak on difficult inputs, which also reduces the contrast between "fixed compute" and "routed compute" in some settings.

7.3 Hard problems in our system: restricted solver budget

In our routed system, hard problems were intentionally restricted to only one Solver call to save tokens and runtime. This constraint reduces cost, but it also limits the main advantage of multi-agent systems on hard inputs: trying multiple solution paths and selecting the best one. After we reduced prompt overhead (Section 6), a natural next step is to allow additional Solver calls only for hard/very-hard cases, which should improve accuracy while keeping average cost controlled.

7.4 Potential bias: diagram-dependent questions evaluated without images

Diagram dependence and interpretation. A major failure mode in our main 150-question run is missing visual information: many questions reference diagrams, but images were not provided to the models. This can systematically reduce accuracy and also distort routing decisions (e.g., difficulty estimation) because key constraints are absent. To partially control for this factor, we include a supplementary 100-question text-only sanity check (Section 6.7), where the advanced system outperforms the baseline (41.0% $\rightarrow$ 47.0% under re-evaluated scoring). These supplementary results do not replace the main experiment, but they strengthen the claim that the routing idea can help even without multimodal inputs.

A practical source of error comes from dataset format. Some questions include figures (e.g. geometry diagrams), but our evaluation did not pass image content into the model input. For such questions, text-only input can be incomplete, leading to systematic failures that are not purely reasoning

errors. The figure below shows an example where the prompt references a diagram (img_3276), but the model did not receive it. This can bias both the measured accuracy and the routing signal (difficulty estimation).

7.5 Takeaway

Overall, the dominant failure drivers in our current setup are (i) strategy-level errors on hard problems under a single-solver budget and (ii) missing visual information for diagram-dependent questions. These observations suggest two clear improvements: selectively increasing solver attempts for hard cases after prompt compression, and incorporating image inputs when the dataset provides diagrams.

Model Input

Q (index=1). In the diagram, $\angle ABF = 41°$, $\angle CBF = 59°$, $DE \parallel BF$, and $EF = 25$. If $AE = EC$, determine the length of AE, to 2 decimal places.

Note: the original question includes a diagram (`img_3276`), but our evaluation used text-only input.

Model Output

Returned answer: `20.83` ✗

Ground truth: `19.14`

Run metadata (logged fields)

Route	BASELINE
Elapsed time	16.86 s
Solver calls	1
Critic used	yes
Judge used	yes
Planner used	no
Summarizer used	no
A-base difficulty (logged)	MEDIUM (confidence 0.75)
A-base provisional answer (logged)	20.83

8 Contributions of group members

- Alan Zhang: took the lead in designing the multi-agent architecture and routing strategy, wrote and iterated on the prompts for all agents, and coordinated the integration of the components into a coherent system.
- Xinmeng Tan: implemented the pipeline in Python, including agent orchestration and logging, and ran experiments and debugging to test the system end-to-end.
- Young Liu: conducted literature review and data / resource collection, and assisted with system design decisions and fine-grained prompt and workflow refinements.

In addition, all members actively contributed ideas to the initial system design; through multiple rounds of discussion and consolidation, the group jointly finalized the baseline and advanced versions of our multi-agent pipeline.

9 Conclusion

In this project, we studied the efficiency–accuracy trade-off of multi-agent math reasoning under a realistic compute budget. We compared a fixed 3-agent baseline (Solver → Critic → Judge) against an advanced system that uses an A-base front-end and a deterministic router to allocate different amounts of computation by estimated difficulty, together with a model-mix strategy. On 150 OlympiadBench questions, the advanced system achieved higher accuracy than the baseline (30% → 36%) while reducing the average runtime (31.54 s → 25.17 s) and the estimated dollar cost (approximately \$ 5 → \$ 1). After auditing and revising our prompts to reduce unnecessary instructions, we further reduced total tokens and observed a small accuracy improvement in re-evaluation (36.0% → 38.67%).

Several things were surprisingly difficult. First, prompt overhead mattered more than expected: input tokens dominated total token usage, so reducing agent calls alone did not automatically reduce token cost unless prompts were also compressed. Second, many failures on hard problems were the early choice of an unproductive approach. Because we limited hard cases to one Solver call to control cost, the system often lacked the ability to "restart" with an alternative solution path. Third, some dataset questions are diagram-dependent, but our evaluation used text-only input, which can systematically hurt accuracy and distort difficulty routing for those examples.

As a late-stage sanity check on a 100-question text-only subset, the baseline (GPT-4.1) achieves 41.0% re-evaluated accuracy and the advanced system achieves 47.0% (+6.0 pp).

For future work, we would pursue three directions. (i) After prompt compression, allow additional Solver attempts only for hard/very-hard routed cases to better address strategy-level failures while keeping average cost controlled. (ii) Incorporate image inputs for diagram-dependent questions to reduce evaluation bias and better match the dataset's intended information. (iii) Evaluate on a larger and more diverse sample to reduce statistical uncertainty and more clearly identify when additional multi-agent computation is worth the cost.

10 AI Disclosure

- Did you use any AI assistance to complete this proposal? If so, please also specify what AI you used.
  - Yes, we use AI assistance. We use ChatGPT to help organize the structure of the final report, optimize English expression, adjust formatting, and assist in reading literature materials.
- If you answered yes to the above question, please complete the following as well:
  - If you used a large language model to assist you, please paste *all* of the prompts that you used below. Add a separate bullet for each prompt, and specify which part of the proposal is associated with which prompt.
    - Summarize the introduction of this paper, and tell me what is the main.
    - Optimize English expressions and adjust the format.
  - Free response: For each section or paragraph for which you used assistance, describe your overall experience with the AI. How helpful was it? Did it just directly give you a good output, or did you have to edit it? Was its output ever obviously wrong or irrelevant? Did you use it to generate new text, check your own ideas, or rewrite text?
    - The AI's responses were not obviously wrong or irrelevant, but some sentences were too general or something wrong and we needed to double check to prevent this situation.
    - We mainly used it to check whether our own ideas are feasible, reorganize sentences, and rewrite text for better readability, rather than to generate new research content.

References

Chang, E. Y. (2025). Maci: Multi-agent collaborative intelligence for adaptive reasoning and temporal planning. arXiv preprint, version 2 (29 Jan 2025).

Chen, J., Zhao, J., Zhao, W.-F., Qin, Y.-F., and Ji, H. (2024). Robust multi-agent communication via diffusion-based message denoising. *IEEE Access*, 12:170437–170450.

Chen, Y., Wang, Y., Zhu, S., Yu, H., Feng, T., Zhang, M., Patwary, M., and You, J. (2025). Multi-agent evolve: Llm self-improve through co-evolution. arXiv preprint, version 3 (last revised 30 Oct 2025).

Derouiche, H., Brahmi, Z., and Mazeni, H. (2025). Agentic ai frameworks: Architectures, protocols, and design challenges. arXiv preprint, version 1 (13 Aug 2025).

Gupta, S., Banerjee, I., and Bhattacharyya, S., editors (2022). *Multi Agent Systems: Technologies and Applications towards Human-Centered*. Springer Tracts in Human-Centered Computing. Springer Singapore, Singapore.

He, C., Luo, R., Bai, Y., Hu, S., Thai, Z. L., Shen, J., Hu, J., Han, X., Huang, Y., Zhang, Y., Liu, J., Qi, L., Liu, Z., and Sun, M. (2024). Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems.

Klesel, M. and Wittmann, H. F. (2025). Retrieval-augmented generation (rag). *Business & Information Systems Engineering*, 67(4):551–561.

Levonian, Z., Li, C., Zhu, W., Gade, A., Henkel, O., Postle, M.-E., and Xing, W. (2023). Retrieval-augmented generation to improve math question-answering: Trade-offs between groundedness and human preference. arXiv preprint, version 2 (11 Nov 2023).

Shao, Z., Wang, P., Zhu, Q., Xu, R., Song, J., Bi, X., Zhang, H., Zhang, M., Li, Y. K., Wu, Y., and Guo, D. (2024). Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint, version 3 (27 Apr 2024).

Wang, H., Wang, R., Xue, B., Xia, H., Cao, J., Liu, Z., Pan, J. Z., and Wong, K.-F. (2024). Appbench: Planning of multiple apis from various apps for complex user instruction. arXiv preprint (v1, submitted 10 Oct 2024).

Zhang, S. and Xiong, D. (2025). Debate4math: Multi-agent debate for fine-grained reasoning in math. In Che, W., Nabende, J., Shutova, E., and Pilehvar, M. T., editors, *Findings of the Association for Computational Linguistics: ACL 2025*, pages 16810–16824, Vienna, Austria. Association for Computational Linguistics.

Zhang, Y., Liu, X., and Xiao, C. (2025). Metaagent: Automatically constructing multi-agent systems based on finite state machines. arXiv preprint, version 1 (30 Jul 2025).

Zhu, Y., Qi, Y., Wang, Z., Gu, L., Sui, D., Hu, H., Zhang, X., He, Z., He, J., Ma, L., and Yu, L. (2025). Healthflow: A self-evolving ai agent with meta planning for autonomous healthcare research. arXiv preprint, version 2 (11 Oct 2025).